

Calibrate Your C Code

A seemingly minor change in how a program is written can bring about a major performance up-grade or vice versa. This article discusses a few techniques to write efficient and optimised C or C++ code.

Any efficient code requires: (a) the selection of the best algorithms and data structures, and (b) code written in a way that the compiler can effectively optimise and generate the best executable.

Ideally, any standard compiler takes whatever code you supply to it and tries to generate the most optimised executable possible, on its own. However, this is not the best practice—a very limited transmutation happens because of many blockers. And prominent among these optimisation blockers is machine-dependency—that is, its dependency on the low-level architectural details of the machine.

Optimising memory references

Performance can depend heavily on many lineaments of the processor design. Therefore, to maximise the performance of a program, both the programmer and the compiler need to understand the target architecture. Understanding the instruction set is another effective way to find out how the code will run, i.e., how frequently memory is referenced and how effectively the machine registers are

being used. Let's consider the following code snippet:

```
void foo (int *x, int *y)
{
    *x = *y;
    *x += *y;
    return;
}

void opt_foo(int *x, int *y)
{
    *x = 2 * *y;
    return;
}
```

Although both the above functions (*foo* & *opt_foo*) have identical behaviour, *opt_foo* is more efficient. Let's see how.

The function *foo* requires five memory references (one read of **x*, two reads of **y*, and two writes of **x*), whereas *opt_foo* just requires two memory references (one read of **y*, and one write of **x*).

Avoiding unwarranted function calls

Another performance blocker may happen due to unwarranted calls to

functions. As an example, you can consider the following two functions:

```
void foo(char *str)
{
    int ctr;
    for(ctr = 0; ctr <= strlen(str); ctr++)
        print("%c", str[ctr]);
}

void opt_foo(char *str)
{
    int ctr, len = strlen(str);
    for(ctr = 0; ctr <= len; ctr++)
        print("%c", str[ctr]);
}
```

Which of the lines of code do you think is better?

foo() calls the library function *strlen()* multiple times. However, irrespective of achieving a similar result, *opt_foo()* calls *strlen()* just once. Here's how a function call works...

Each function code runs because of an immediate change in the value of the program counter (*pc*). But before jumping to call the function from its current state, the current value of *pc*, along with other values, needs to be